Scale

NestUp

Heavy queries, indexes, caching, measured results, and limits

Internet Technologies — Become a Full-Stack Engineer

RUNI CS 2026 · Final project

Live product **nestup-kappa.vercel.app**

Repository **github.com/nestupweb/nestup**

Current production data: 845 profiles · 824 listings · 1,290 household memberships · 883 listing views · 29 conversations **Database:** Postgres 17, Supabase (eu-central-1) · **Hosting:** Vercel serverless

---

1. Where the product stands today

This should be stated first, because it changes what the honest answers below actually are. The dataset is not a toy dataset. The seed contains 838 accounts and 824 live rooms distributed across 124 cities, which is approximately the size of the first city of a real product. Therefore the queries described in this document are executed against realistic row counts and not against three test records, and all of the measurements presented here were taken from the live deployment.

---

2. What happens as users arrive

Tens of concurrent users

There is no problem in this range, and no change is required. Vercel executes each request inside its own serverless invocation, and consequently concurrency is handled by the platform rather than by a process pool. The database remains well inside the pooled connection limits of Supabase. The heaviest read, which is the Listings index, is a **shared cache entry** that is served identically to every user.

Hundreds of concurrent users

This range is still acceptable, although there is one reservation which is worth naming explicitly.

The following aspects hold up well:

- **The public room list scales with the amount of content and not with the amount of traffic.** The function `queryListings` is a shared `"use cache"` entry keyed on the filter set, and therefore a thousand people who open Listings with the default filters cause **one** database query rather than a thousand.
- **Per-member data never reaches a shared store.** Decks, profile tabs, saved identifiers and inboxes are all declared as `"use cache: private"` and are held inside the browser of each individual member. They therefore cost nothing on the server and cannot contend with one another.
- **Pages send HTML rather than producing a waterfall of client-side fetches.**

The following aspects would show strain first:

- **Realtime connections.** Every open chat holds a WebSocket. The free tier of Supabase limits the number of concurrent connections, and this is the first ceiling which the product would reach, before CPU, before storage and before query time.

- **Deck construction.** This is discussed in §3. It is the single genuinely expensive read, and it cannot be shared between members.

Thousands of concurrent users

This range requires the work described in §8. Principally, the deck must cease to perform its scoring in application code, and Realtime requires a concrete plan.

---

3. Heavy queries, and what was done about them

3.1 The swipe deck, which is by far the most expensive

Building the deck of a single member originally required **approximately 9 round-trips arranged in 3 dependent waves**: first the candidate rooms, then the owners and the residents, and finally the blocked identifiers and the attention history. Every one of these queries is scoped to one specific member, and therefore nothing can be shared between members.

Three mechanisms keep this acceptable:

1. **The hard filters are executed in SQL, before any scoring takes place.** The city filter, `is_active`, `removed_at is null` and the exclusion of already-swiped rooms are all evaluated inside Postgres. Only the surviving rows are scored. The naive alternative would be to score all 824 rooms inside Node for every single deck view.
2. **A cap on the deck size**, namely `DECK_SIZE = 60`.
3. **The entire result is cached per member** through `getCachedDeck`, tagged as `deck:<userId>`, and it is invalidated only when the member swipes or modifies a room belonging to that member.

An honest limitation: the scoring itself is JavaScript executed over the filtered candidate set. This is acceptable with 824 rooms in a single city, but it is not acceptable with 50,000 rooms, and §8 describes what would replace it.

3.2 The Listings index

This query is `select * from listings where is_active`, combined with the filters, together with `count: 'exact'` in order to support pagination.

- It is backed by `listings_browse_idx on (city, rent, available_from) where is_active`, which is a **partial** index and therefore covers only those rows which are actually capable of appearing in the result.
- It is paginated at 20 rows per page, and the page is applied inside SQL through `range()` and never inside JavaScript.
- It is stored in the shared cache, and consequently repeated traffic is free.

An honest limitation: the option `count: 'exact'` causes Postgres to count every matching row on every uncached query, and this degrades as the table grows. The standard remedy is an estimated count. It was deferred because at the present size, the fact that "824 rooms available" is exactly correct is worth more than the milliseconds which would be saved.

3.3 The map, which loads all pins at once

The endpoint `/api/listings/pins` returns **every** placed room, which is approximately 824 rooms and about 150 KB. This is a deliberate product decision, since the map is intended to display everything rather than the current filter.

The cost is contained by paying it only when it is genuinely required:

- The endpoint selects **only** the columns `id, lat, lng, rent, title, city, neighborhood, photo_urls[0]`, and it does not perform `select *`.
- **No map is rendered until an icon is pressed.** Both MapLibre and the pin payload are loaded on the first opening of the map and are then retained for the remainder of the visit. The majority of visits never open a map and therefore never pay this cost at all.

3.4 The chat inbox

The function `my_conversations()` is a single SQL function which returns every thread together with its last message, its unread count and the identity of the other party. It therefore requires one round-trip rather than one query per conversation.

It is backed by `messages_conversation_idx (conversation_id, created_at)`.

3.5 Household size and gender

These values are required on every row of the browse list and on every deck candidate. Computing them per row would transform a single query into hundreds of queries, and therefore they are **denormalized onto `listings`** and maintained by triggers over `listing_residents`. This is the classic trade of write cost in exchange for read benefit, and in this case it is oriented in the correct direction, since rooms are read far more frequently than households change.

---

4. Indexes

There are **20 indexes**, and each one of them is placed against a query which actually exists. The following are the indexes which carry the product:

INDEX	WHAT IT SERVES
`listings_browse_idx (city, rent, available_from) WHERE is_active`	The Listings index and the hard filter of every deck. It is partial, and therefore inactive rooms are not present in it
`one_active_listing_per_owner (owner_id) WHERE is_active`	**Unique.** It enforces the rule of "one person, one home" as a constraint rather than as a check
`swipes_by_seeker_idx (seeker_id)`	The exclusion of already-swiped rooms, which is read on every deck build
`swipes_likes_by_listing_idx (listing_id) WHERE direction = 'like'`	Interest counts per room. It is partial, since skips constitute the majority and are never counted
`messages_conversation_idx (conversation_id, created_at)`	Thread history and last-message lookups
`messages_client_id_uniq`	**Unique.** It implements the idempotent retry, and it is the reason that a resend cannot post a message twice
`listing_views_recent_idx (user_id, viewed_at DESC)`	The History section of the profile, with `LIMIT 30`. The `DESC` direction matches the ordering of the query
`viewings_by_conversation_idx (conversation_id, created_at)`	Viewings inside a thread
`saved_listings_by_listing_idx (listing_id)`	Save counts
`blocks_blocked_idx (blocked_id)`	The function `blocked_user_ids()`, which is called on every deck build
`listing_invites_pending_idx (invitee_id) WHERE status = 'pending'`	Pending invitations. It is partial, since invitations which have already been answered are of no further use
`profiles_full_name_trgm_idx`	A **trigram** index for searching roommates by name, because a `LIKE '%name%'` pattern cannot use a B-tree
`listings_household_size_idx`, `listings_household_gender_idx`, `listings_wanted_gender_idx`	The denormalized filter columns

The pattern which is worth pointing out is that **six of these indexes are partial or expression indexes.** A partial index on `WHERE is_active` is smaller and remains resident in cache, precisely because it never contains the rows which the query would be unable to return in any case.

5. Avoiding unnecessary loading

This is the area in which most of the recent engineering effort was invested, and the result is measurable.

5.1 Server-side caching

There are five cached reads, each of which is tagged so that a write invalidates exactly what it modified:

READ	CACHE	TAG	INVALIDATED BY
`queryListings`	**shared**	`listings`	Publishing, editing, pausing or removing a room
`getSavedListingIds`	private	`saved:<id>`	Adding a room to the liked list
`getProfileTabData`	private	`profile:<id>`	Changes to the profile, the listing, the hearts or the invitations
`getCachedDeck`	private	`deck:<id>`	Swiping, and changes to a room owned by the member
`getCachedInbox`	private	`chat:<id>`	Sending, reading or deleting a chat, **and also a message written by the other side, through Realtime**

The last row of this table is the interesting one. The inbox of a member changes when *somebody else* writes a message, and in that situation no action of the member is executed at all. Therefore the Realtime socket calls a Server Action which drops the tag belonging to that member, and this is exactly what makes the caching of an inbox safe rather than stale.

The rule regarding the scope of invalidation. Mutations used to respond to every write with a large number of `revalidatePath` calls, and consequently editing a room also discarded the chats and the profile of the member. At present a write invalidates only what it actually modified, and `cache-invalidation.test.ts` fails if this behavior regresses.

5.2 Measured result

The following figures describe returning to a tab which was already visited, measured on production with a warm cache:

TAB	BEFORE	AFTER
Swipe	860 ms, skeleton	**69 ms, no skeleton**
Chat	1,900 ms, skeleton	**71 ms, no skeleton**
Profile	~370 ms, skeleton	**69 ms, no skeleton**
Listings	~850 ms, skeleton	~375 ms, ~300 ms skeleton

Three of the four tabs make **zero server requests** when the member returns to them.

The technical reason is worth being able to explain. The App Shell prerender of Next.js advances through cached reads and **stops at the first uncached read**. Every page began with an uncached `auth.getUser()` call, which is a network round-trip to Supabase, and this kept everything positioned behind it outside the shell regardless of how well that content was cached. Caching this identity read is therefore what removed approximately 300 ms from each tab.

Listings is the remaining case. It likewise makes no server request, and its residual cost is approximately 60 ms of client rendering, which is extended by the page-slide animation. This is therefore an issue of animation timing and not an issue of data.

5.3 Payload discipline

- **Pagination is applied everywhere that it matters:** Listings uses 20 rows per page in SQL, History uses `LIMIT 30`, and the deck is capped at 60.
- **Column selection:** the pins endpoint selects 8 columns and not `*`.
- **Images:** `next/image` is used with correct `sizes` values, so that a thumbnail of 128 pixels is not delivered as a file of 1200 pixels. Only the first three cover images are marked as `priority`, and the swipe deck warms exactly **one** card in advance rather than the entire deck.
- **Code splitting:** MapLibre is a dynamic import placed behind a button press.
- **Photographs are compressed inside the browser** before they are uploaded.

---

6. Separation between client and server

RUNS ON THE SERVER	RUNS IN THE BROWSER
Every database read (Server Components)	Interaction state, such as dialogs, the current card and filter drafts
Every database write (Server Actions)	Optimistic rendering of messages
All scoring and ranking	Map rendering (MapLibre and WebGL)
All authorization	Image compression prior to upload
Secrets, namely the service-role key, SMTP and Gemini	The Realtime subscription

No credential which is capable of bypassing RLS ever reaches the browser. The client holds the anon key only. That key is designed to be public and is powerless without a session. The service-role key exists exclusively inside server-side environment variables.

Two boundary details are enforced rather than merely assumed:

- The module `lib/supabase/public.ts` is a **cookie-free** client which is used only for the shared cache. A client which carried a session in that position would make the shared cache entry member-

specific, which would constitute a cross-user leak. For this reason the module is marked `server-only` and documented accordingly.

- The modules `lib/swipe.ts`, which is client-safe, and `lib/swipe-deck.ts`, which is `server-only`, are separated precisely because the cookie-reading client must never enter a browser bundle.

---

7. Current limitations

These are stated honestly, and each of them represents a genuine ceiling.

1. **Deck scoring is performed on the application side.** This is acceptable with approximately 824 rooms, but it is the wrong architecture at 50,000 rooms.
2. **The option `count: 'exact'`** on the Listings query counts every matching row whenever the cache misses.
3. **Realtime connection limits** constitute the first hard ceiling on the free tier.
4. **There is no rate limiting except on authentication email.** The table `auth_mail_throttle` protects the mail path, whereas message sending and listing creation are protected only by RLS.
5. **Photographs are checked by an external model on the upload path**, which adds latency and creates a dependency on the availability of a third party.
6. **There are no background jobs.** Everything happens inside the request, and notification emails are sent inline.
7. **There is a single region.** The database is located in eu-central-1, which is appropriate for a product focused on Israel but poor for users elsewhere.
8. **There is no CDN caching of the pins payload.** The 150 KB payload is fetched separately for every visitor who opens the map.
9. **Storage is unbounded.** Nothing prunes the photographs belonging to removed listings.
10. **There is no observability.** There are no metrics, no tracing and no slow-query alerting, and therefore problems would be discovered only when a person happened to notice them.

---

8. What a larger version would change

The following list is ordered according to the sequence in which the pain would actually arrive.

The first step is to measure. Nothing on this list is worth implementing before a slow-query log exists and a p95 latency figure is available per route. Until then, every item below remains a hypothesis.

1. **Move deck filtering entirely into SQL.** The weighted score would be computed as a SQL expression, or alternatively as a materialized per-member candidate table which is refreshed whenever the profile changes, so that Postgres returns 60 ranked rows instead of Node scoring hundreds of candidates. This is the single change which unlocks an order of magnitude of growth.

2. **Estimated counts.** Replace `count: 'exact'` with a planner estimate above a certain threshold, while retaining exact counts for small result sets.
3. **Cursor-based pagination.** The offsets used by `range()` degrade deep inside a long list, whereas keyset pagination on `(created_at, id)` does not.
4. **Move notification email onto a queue.** It should not remain on the request path.
5. **Rate limiting** on message sending, listing creation and report submission.
6. **Cache the pins payload at the CDN edge**, together with a tag-based purge, since that payload is identical for every user.
7. **A plan for Realtime:** either move to a paid tier, or replace always-on sockets with polling for the inbox while reserving sockets for threads which are actually open.
8. **Add observability**, namely Vercel Analytics, Supabase slow-query logs and an error tracker.
9. **A storage lifecycle**, meaning that photographs are deleted when a listing is hard-removed.
10. **Read replicas and multiple regions**, but only if the product ever expands beyond a single country. This item is listed last deliberately, because it is the change which people tend to reach for first and to require last.
